

OpoFit — Documentación para el tribunal (profesorado)

Autor: Rafael Javier Luengo Barreno

Titulación: Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma
(2.º curso)

Proyecto: Aplicación final / Trabajo Fin de Grado

Curso académico: 2025/2026

Presentación de este documento

Este texto va dirigido al **tribunal y al profesorado**: resume el alcance del proyecto, las competencias que intento demostrar, cómo organicé el trabajo y qué resultados obtuve. El detalle de código, API, base de datos y fichas por archivo Kotlin está en la **memoria técnica** (`Documentacion_Completa_OpoFit.md`), que es la que conviene usar para seguir el rastro línea por línea. Aquí el foco es **planificación, metodología, evaluación y conclusiones**, no sustituir a la memoria larga.

Índice general

Núcleo (1–13)

- 1. Resumen ejecutivo
- 2. Competencias del módulo/proyecto (marco DAM)
- 3. Alcance funcional
- 4. Marco teórico mínimo
 - 4.1. Cliente–servidor y API REST
 - 4.2. Autenticación basada en token
- 5. Arquitectura del sistema
 - 5.1. Vista lógica
 - 5.2. Vista física (despliegue)
- 6. Diseño de datos

- **7.** API REST — contrato resumido
- **8.** Lógica de negocio relevante para evaluación
 - **8.1.** Cálculo de nota y nivel
 - **8.2.** Incompletitud de perfil
 - **8.3.** RSS
- **9.** Cliente Android
 - **9.1.** Patrones utilizados
 - **9.2.** Seguridad en cliente
 - **9.3.** Experiencia de usuario
- **10.** Metodología y herramientas
 - **10.1.** Enfoque de desarrollo incremental
 - **10.2.** Control de calidad manual
 - **10.3.** Entorno de pruebas
- **11.** Competencias específicas observables (rúbrica orientativa)
- **12.** Resultados y validación
- **13.** Trabajo futuro

Ampliaciones y material de apoyo (secciones 14 a 38)

- **14.** Planificación temporal del proyecto (orientativa)
 - **14.1.** Hitos de integración
- **15.** Matriz ampliada de competencias DAM (indicadores observables)
 - **15.1.** Indicadores de desempeño (rúbrica orientativa ampliada)
- **16.** Gestión de riesgos técnicos y mitigación
- **17.** Ética, privacidad y uso de datos (marco RGPD, nivel académico)
- **18.** Casos de prueba de aceptación (lista ampliada)
- **19.** Glosario de términos (ampliado)
- **20.** Reflexión del autor sobre aprendizajes
- **21.** Bibliografía y recursos normativos (ampliada)
- **22.** Narrativa de desarrollo por fases (relato técnico)
 - **22.1–22.4.** Fases A–D (semanas 1–16)
- **23.** Guía ampliada para la defensa oral (preguntas frecuentes)

- **24.** Competencias transversales (comunicación, aprendizaje autónomo, digitalización)
- **25.** Indicadores de sostenibilidad del código (autoevaluación)
- **26.** Marco normativo y formativo (Formación Profesional en España)
- **27.** Descripción funcional de pantallas (vista tribunal)
- **28.** Rúbrica extensa de evaluación (orientación para el tribunal)
 - **28.1–28.4.** Criterios (análisis y diseño; backend; Android; documentación)
- **29.** Bibliografía y fuentes comentadas (ampliada)
- **30.** Anexo: guion sugerido de demostración (15–20 minutos)
- **31.** Relación explícita con el documento técnico completo
- **32.** Diario de trabajo por semanas (plantilla desarrollada)
 - **32.1.** Reflexión sobre el reparto de esfuerzo
- **33.** Cuestionario extenso para preparar la defensa (respuestas modelo)
- **34.** Léxico inglés–español (términos del proyecto)
- **35.** Comparativa de alternativas tecnológicas (y decisión)
- **36.** Figuras y anexos gráficos recomendados para la memoria impresa
- **37.** Mensaje final para el tribunal (lectura rápida)
- **38.** Microguía: 24 preguntas breves con respuesta en una frase

Cierre (secciones 39 a 41)

- **39.** Conclusión académica
 - **40.** Referencias técnicas (consulta)
 - **41.** Cómo se complementan los dos documentos
-

1. Resumen ejecutivo

OpoFit es un sistema software compuesto por:

1. **API REST** desarrollada en **Node.js** y **Express 5**, con autenticación basada en **JWT**, integración con **Google OAuth** (verificación de token en servidor) y **Firebase Authentication** (verificación de ID token con **Firebase Admin SDK**).
2. **Base de datos relacional MySQL** que modela oposiciones, pruebas físicas oficiales, baremos, rutinas predefinidas, rutinas personalizadas, historial de sesiones y resultados por ejercicio.

3. **Ciente Android** en **Kotlin**, interfaz **Jetpack Compose**, consumo de API mediante **Retrofit** y persistencia local de sesión con **DataStore**.

El objetivo pedagógico principal es demostrar la integración de competencias de **acceso a datos, desarrollo de interfaces, programación multimedia y dispositivos móviles** y **despliegue de servicios en red**, en un producto coherente.

2. Competencias del módulo/proyecto (marco DAM)

Competencia / área	Evidencia en el proyecto
Almacenamiento e integridad de datos	Esquema MySQL normalizado, claves foráneas, transacciones en operaciones multi-tabla
Servicios en red	API REST versionada bajo /api, JSON, código HTTP semántico
Seguridad	Hash bcrypt, JWT, rate limiting, comprobación de propiedad de recursos (userId vs token)
Interfaz de usuario	Material Design 3, Compose, navegación declarativa
Calidad	Pruebas unitarias/integración en backend (Jest), separación rutas–controladores–servicios

3. Alcance funcional

- Registro e inicio de sesión (email/contraseña, Google, Firebase).
- Selección de **oposición** y registro de **marcas** por prueba oficial.
- **Cálculo automático** de un nivel orientativo (BÁSICO / INTERMEDIO / AVANZADO) a partir de los baremos almacenados.
- **Asignación de rutina oficial** segmentada por **enfoque** (fuerza, velocidad, resistencia) según tablas rutinas_opo y detalle_rutina_opo.
- **Rutinas personalizadas** a partir del catálogo de ejercicios.
- **Historial de entrenamiento** con registro de resultados por ejercicio y consulta de evolución.
- **Información de oposición**: textos de pruebas, requisitos por nivel, agregación de **noticias RSS** de fuentes públicas (BOE, instituciones).

Fuera de alcance explícito: panel web administrativo, notificaciones push en tiempo real, sincronización multicliente concurrente avanzada.

4. Marco teórico mínimo

4.1. Cliente-servidor y API REST

En arquitecturas **cliente-servidor**, el cliente (app móvil) solicita recursos o acciones al servidor mediante protocolos estándar. **REST** (Representational State Transfer) organiza esos recursos con **URLs**, **verbos HTTP** y representaciones (habitualmente **JSON**). Ventajas para un TFG de DAM: desacoplamiento (se puede cambiar el cliente sin reescribir toda la lógica), testabilidad del backend con herramientas como Postman o Jest, y alineación con prácticas de la industria.

4.2. Autenticación basada en token

El servidor no mantiene sesión en memoria por cada usuario: tras el login emite un **JWT** firmado con secreto. El cliente reenvía el token en cada petición; el servidor **verifica la firma** y extrae el identificador de usuario. Esto escala mejor que sesiones de servidor clásicas en PHP con `session_start()`, aunque exige proteger el secreto y usar **HTTPS** en producción.

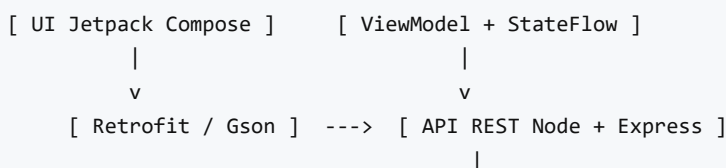
Petición autenticada (el Bearer es el JWT del login):

```
GET /api/auth/me HTTP/1.1
Host: localhost:3000
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Accept: application/json
```

5. Arquitectura del sistema

5.1. Vista lógica

El sistema sigue un patrón **cliente-servidor**:



- El **cliente Android** no accede directamente a MySQL; toda persistencia pasa por la API.
- El **servidor** concentra reglas de negocio (cálculo de nivel, restricción de un entreno por día y rutina, etc.).
- **Terceros** (Google, Firebase, fuentes RSS) se consumen desde el servidor o desde el cliente según el flujo (tokens OAuth/Firebase validados en backend).

5.2. Vista física (despliegue)

- Base de datos MySQL en instancia local o servicio gestionado.
- Backend Node desplegable en plataforma PaaS (en el código Android aparece URL de producción tipo **Railway** en BuildConfig).
- Cliente Android distribuible como APK/AAB firmado.

6. Diseño de datos

Se utiliza un modelo **entidad-relación** con integridad referencial estricta. Las entidades nucleares son:

- **Oposición** 1:N **Pruebas oficiales** 1:N **Baremos** (puntuación por umbral de marca y género).
- **Usuario** 1:1 **Settings** (unidades); 1:N **Marcas de perfil** (mejor marca por prueba); 1:N **Historial de sesiones**; 1:N **Rutinas personalizadas**.
- **Rutina oficial** N:M **Ejercicios** mediante **detalle_rutina_opo** (series, repeticiones, descanso).
- **Historial** 1:N **Registro de resultados** por ejercicio completado en sesión.

El archivo BBDD/mydb.sql define el DDL; seed.sql aporta datos de prueba coherentes con la aplicación.

7. API REST — contrato resumido

Todos los endpoints bajo /api devuelven JSON. Los endpoints protegidos requieren cabecera:

Authorization: Bearer <JWT>

Grupos principales:

- **/api/auth:** registro, login, flujos Google/Firebase, /me.
- **/api/user:** actualización de perfil y preferencias, eliminación de cuenta.
- **/api/oposiciones:** catálogo, detalle, requisitos, RSS.
- **/api/rutinas:** cálculo de entrenamiento personalizado según marcas.
- **/api/rutinas-pers:** CRUD de rutinas libres.
- **/api/historial:** registro y consultas de progreso.
- **/api/info:** información de baremos y marcas del usuario.
- **/api/ejercicios:** catálogo global.

Se aplica **rate limiting** global y reforzado en rutas de autenticación para mitigar abuso.

Login por email y respuesta típica:

```
{ "email": "opofit.demo@local", "password": "TuContraseñaSegura" }
```

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": { "id": 1, "email": "opofit.demo@local", "nombre": "Demo" }
}
```

8. Lógica de negocio relevante para evaluación

8.1. Cálculo de nota y nivel

A partir de las marcas del usuario y la tabla baremos_puntuacion, se obtiene la **mejor nota** posible por prueba respetando el sentido del test (mejor_si_es_menor). La **media** de notas determina el nivel sugerido según umbrales definidos en RutinasService.

8.2. Incompletitud de perfil

Si faltan pruebas por registrar para la oposición, el sistema **no asigna nivel** y comunica estado **INCOMPLETO** al cliente, evitando conclusiones erróneas.

8.3. RSS

El servicio `RssService` agrega feeds externos, maneja errores de red y formatos heterogéneos (RSS 2.0, Atom), y devuelve una lista unificada para la capa de presentación.

9. Cliente Android

9.1. Patrones utilizados

- **MVVM:** `ViewModel` + `StateFlow` para estado reactivo.

Ilustración mínima: la pantalla observa el estado y reacciona sin mezclar lógica de red en el `Composable`.

```
// Idea: el ViewModel expone estado; la UI solo colecciona y llama a acciones
val uiState by miViewModel.state.collectAsState()
```

- **Repository implícito:** servicios `Retrofit` llamados desde `ViewModels`; autenticación encapsulada en `BackendAuthService`.
- **Single Activity:** `MainActivity` con `NavHost Compose`.

9.2. Seguridad en cliente

- Almacenamiento de token en **`DataStore`** (preferible a `SharedPreferences` para operaciones asíncronas).
- JWT enviado solo en cabecera; no se expone en logs de usuario.

9.3. Experiencia de usuario

- Material 3, soporte de tema claro/oscuro persistido.
 - Mensajes de error derivados de códigos HTTP (401, 409, etc.) en `HistorialViewModel` y similares.
-

10. Metodología y herramientas

- **Control de versiones:** Git (recomendado documentar ramas y commits significativos en la memoria final).
- **IDE:** Android Studio, Visual Studio Code o similar para Node.
- **Pruebas backend:** Jest con mocks de base de datos.
- **Documentación:** memoria en Markdown y PDF para la entrega académica.

10.1. Enfoque de desarrollo incremental

El trabajo se ha abordado por **capas verticales**: primero esquema de datos y semillas mínimas; después autenticación y usuario; a continuación rutinas oficiales y cálculo de nivel; por último rutinas personalizadas, historial y capa RSS. Esta estrategia reduce el riesgo de integración tardía y permite enseñar software “usable” en cada hito.

10.2. Control de calidad manual

Además de Jest, se han ejecutado **pruebas manuales** sobre emulador y/o dispositivo físico: registro, login, flujo de entreno completo, registro duplicado de sesión (caso 409), cambio de unidades en ajustes y verificación de persistencia tras reiniciar la app.

10.3. Entorno de pruebas

- **Backend:** Node en máquina local o PaaS; base MySQL poblada con `seed.sql`.
- **Android:** emulador Pixel API 34+ recomendado; comprobar también una versión mínima cercana a `minSdk` si el centro exige matriz de compatibilidad.

11. Competencias específicas observables (rúbrica orientativa)

Indicador	Evidencia en OpoFit
Modelado de datos relacional	Esquema <code>mydb.sql</code> con FK e índices
API REST documentada	Prefijos <code>/api</code> , JSON, códigos HTTP coherentes
Autenticación y autorización	JWT + comprobación de <code>userId</code> en rutas sensibles
Cliente móvil nativo	Kotlin + Compose + ViewModels

Indicador	Evidencia en OpoFit
Integración servicios externos	Google OAuth, Firebase Admin, RSS
Pruebas automatizadas	Carpeta backend/tests con Jest

12. Resultados y validación

Criterios de aceptación sugeridos para la defensa oral:

- Registro e inicio de sesión completados sin errores críticos.
- Cálculo de nivel coherente con datos de `seed.sql`.
- Creación y ejecución de rutina personalizada con registro en historial.
- Consumo de endpoint RSS con respuesta no vacía en condiciones de red normales.

Limitaciones conocidas: dependencia de disponibilidad de feeds RSS externos; variaciones en latencia de red móvil.

13. Trabajo futuro

- Tests UI (Compose) y tests de integración end-to-end.
- Panel de administración para baremos sin tocar SQL.
- Internacionalización completa (i18n).
- Mapa de progreso visual avanzado (gráficas).

14. Planificación temporal del proyecto (orientativa)

La siguiente tabla resume un **calendario tipo** de TFG en DAM (16 semanas). Los hitos no son fechas contractuales: sirven para explicar al tribunal **cómo se distribuyó** el esfuerzo entre análisis, implementación y validación.

Semana	Hito principal	Entregable observable
1–2	Definición del problema, estado del arte breve, alcance	Documento de requisitos iniciales

Semana	Hito principal	Entregable observable
3-4	Modelo de datos y script SQL	mydb.sql + seed.sql revisados
5-6	Autenticación (registro, login, JWT)	Endpoints /api/auth + cliente login
7-8	Rutinas oficiales y cálculo de nivel	RutinasService + pantallas de entreno
9-10	Rutinas personalizadas y catálogo ejercicios	CRUD rutinas libres + UI
11-12	Historial y evolución	ProgresoService + gráficas/listas en app
13-14	Integraciones (Google, Firebase, RSS)	Flujos probados en dispositivo
15	Pruebas Jest, pruebas manuales, corrección de bugs	Informe de pruebas breve
16	Redacción de memoria, exportación PDF, preparación defensa	Paquete de entrega

14.1. Hitos de integración

1. **Primera vertical completa:** usuario registrado que consulta oposiciones (lectura).
2. **Segunda vertical:** login con token y llamada autenticada a perfil o rutinas.
3. **Tercera vertical:** registro de sesión de entrenamiento y consulta de historial.
4. **Cuarta vertical:** rutina libre creada, ejecutada y reflejada en progreso.

Cada hito reduce el riesgo de acumular deuda técnica invisible hasta la fecha límite.

15. Matriz ampliada de competencias DAM (indicadores observables)

Ámbito curricular	Qué se espera evaluar	Evidencia concreta en OpoFit
Programación orientada a objetos / modularidad	Código separado por responsabilidades	Rutas → controladores → servicios en Node; paquetes data, ui, domain en Android

Ámbito curricular	Qué se espera evaluar	Evidencia concreta en OpoFit
Acceso a datos	SQL coherente, transacciones donde procede	Inserciones relacionadas en registro de usuario; historial con detalle de resultados
Desarrollo de interfaces	Navegación clara, feedback de errores	NavHost, pantallas Compose, mensajes ante 401/409
Servicios en red	API REST, JSON, seguridad básica	JWT, rate limiting, CORS acotado
Despliegue / entorno	Configuración externa vía variables	DB_*, JWT_SECRET, Firebase service account
Calidad y pruebas	Automatización mínima razonable	Jest en backend/tests

15.1. Indicadores de desempeño (rúbrica orientativa ampliada)

Nivel	Criterio
Satisfactorio	La app cumple flujos principales; la API responde con códigos HTTP adecuados; existe documentación básica.
Notable	Hay separación de capas clara; pruebas automáticas en puntos críticos; modelo de datos normalizado.
Sobresaliente	Integración de terceros documentada; manejo de errores de red y sesión; reflexión crítica sobre limitaciones.

16. Gestión de riesgos técnicos y mitigación

Riesgo	Impacto	Probabilidad	Mitigación aplicada
Pérdida de sesión por token caducado	Alto	Media	Endpoint /me al iniciar; mensaje guiado al login
Feeds RSS caídos o lentos	Medio	Alta	Timeouts y manejo de errores en RssService; UI tolerante a listas vacías

Riesgo	Impacto	Probabilidad	Mitigación aplicada
Inconsistencia de baremos	Alto	Baja	Datos en tablas versionables; validación en servicio antes de nivel
Sobrecarga de API (fuerza bruta)	Medio	Media	Rate limiting reforzado en auth
Errores de integración Google/Firebase	Alto	Media	Verificación en servidor; no confiar solo en el cliente

17. Ética, privacidad y uso de datos (marco RGPD, nivel académico)

El tratamiento de datos personales (email, nombre, datos físicos) debe alinearse con **minimización** y **finalidad**: solo se recogen campos necesarios para el cálculo de nivel y la experiencia de entreno. En un entorno académico:

- Los datos de demostración en `seed.sql` son ficticios o anonimizados.
- En producción real sería obligatorio informar al usuario en política de privacidad, base legal, derechos ARCO+ y medidas de seguridad (HTTPS, rotación de secretos).
- Los tokens de Google/Firebase se validan en servidor para no aceptar identidades falsificadas.

Esta sección no sustituye el asesoramiento legal: documenta la **conciencia** del estudiante sobre el contexto normativo.

18. Casos de prueba de aceptación (lista ampliada)

Cada caso incluye **precondición**, **acción** y **resultado esperado** para facilitar una demo reproducible ante tribunal.

1. CA-01 Registro email

- Pre: app instalada, backend y BD operativos.
- Acción: completar formulario válido y confirmar.
- Esperado: usuario creado; redirección o acceso a flujo principal; token almacenado.

2. CA-02 Login email

- Pre: usuario existente.
- Acción: credenciales correctas.
- Esperado: 200 en login; token presente en DataStore.

3. CA-03 Login credencial incorrecta

- Pre: usuario existente.
- Acción: contraseña errónea.
- Esperado: mensaje de error claro; no token válido.

4. CA-04 Google / Firebase (si configurado)

- Pre: cuenta de prueba.
- Acción: flujo nativo de proveedor.
- Esperado: backend verifica token; sesión coherente con email mostrado.

5. CA-05 Selección de oposición y marcas

- Pre: usuario autenticado.
- Acción: introducir marcas según pruebas de la oposición.
- Esperado: cálculo de nivel o estado INCOMPLETO si faltan pruebas.

6. CA-06 Rutina oficial por enfoque

- Pre: perfil completo.
- Acción: solicitar entrenamiento (fuerza/velocidad/resistencia).
- Esperado: bloques y ejercicios coherentes con tablas de rutina.

7. CA-07 Rutina personalizada

- Pre: usuario autenticado.
- Acción: crear rutina con ejercicios del catálogo.
- Esperado: persistencia en BD; listado actualizado.

8. CA-08 Registrar entrenamiento

- Pre: rutina disponible.
- Acción: completar formulario de sesión y resultados por ejercicio.
- Esperado: fila en historial; puntos en evolución.

9. CA-09 Duplicado de sesión mismo día (si aplica regla)

- Pre: sesión ya registrada.
- Acción: intentar segundo registro conflictivo.
- Esperado: HTTP 409 o mensaje equivalente en cliente.

10. CA-10 Ajustes y persistencia

- Pre: usuario autenticado.
- Acción: cambiar unidades/tema; cerrar y reabrir app.
- Esperado: preferencias conservadas.

11. CA-11 RSS

- Pre: red disponible.
- Acción: abrir información de oposición con noticias.
- Esperado: lista o mensaje de indisponibilidad sin crash.

12. CA-12 Cierre de sesión

- Pre: usuario autenticado.
- Acción: logout.
- Esperado: token borrado; pantalla de login.

19. Glosario de términos (ampliado)

Término	Definición breve
API REST	Estilo de arquitectura web basado en recursos, verbos HTTP y representaciones como JSON.
JWT	Token firmado que transporta claims; en OpoFit identifica al usuario autenticado.
bcrypt	Algoritmo de hash para contraseñas con factor de trabajo configurable.
Compose	Toolkit declarativo de UI de Android; el árbol de UI es una función de estado.
Corrutina	Concurrency ligera en Kotlin para operaciones asíncronas sin bloquear el hilo principal.
DataStore	Almacenamiento tipado de preferencias; sustituto moderno de SharedPreferences para muchos casos.

Término	Definición breve
MVVM	Model-View-ViewModel: separación entre UI, estado de presentación y datos.
InnoDB	Motor transaccional de MySQL usado en el proyecto para integridad referencial.
Baremo	Tabla que traduce una marca física en puntuación o categoría.
RSS	Formato de sindicación de contenidos usado para agregar noticias externas.
Rate limiting	Limitación de frecuencia de peticiones para mitigar abusos.
OAuth / OpenID	Marcos de autorización; aquí se usa verificación de tokens de Google.
Firebase Admin SDK	Librería servidor para verificar ID tokens emitidos por Firebase Auth.

20. Reflexión del autor sobre aprendizajes

Durante el desarrollo de OpoFit se consolidaron competencias transversales: **lectura de documentación oficial**, **depuración sistemática** (tanto en cliente como en servidor) y **pensamiento en datos** (normalización y coherencia entre tablas). La integración de autenticación con terceros mostró la importancia de **no confiar en el cliente** y de centralizar la verificación en backend. Asimismo, la experiencia con Compose refuerza la idea de **estado unidireccional** y pruebas de regresión manuales en flujos largos (registro → entreno → historial).

Las principales **dificultades** encontradas suelen estar en la frontera red/servidor (latencia, errores intermitentes) y en la variabilidad de fuentes RSS. Reconocer estas limitaciones forma parte de la madurez técnica esperada en un TFG de grado superior.

21. Bibliografía y recursos normativos (ampliada)

- Ministerio de Educación y Formación Profesional — ordenación de los ciclos formativos de grado superior (consultar versión vigente en el BOE).
- Documentación **Express**, **mysql2**, **Jest**, **Retrofit**, **Jetpack Compose**, **Navigation Compose**, **DataStore**.
- **MySQL 8.0 Reference Manual** — sintaxis SQL, InnoDB, restricciones.

- **RFC 7519** — JSON Web Token (visión general).
 - Guías de **Material Design 3** para coherencia visual.
-

Fin del bloque ampliado (secciones 14–21).

22. Narrativa de desarrollo por fases (relato técnico)

Esta sección no sustituye un diagrama de Gantt formal: ofrece una **narrativa** que el tribunal puede seguir cuando pregunte “¿cómo empezó todo?”. La idea es mostrar **orden** y **priorización**.

22.1. Fase A — Del problema al modelo (semanas 1–4)

En una primera iteración se delimitó el **problema**: los opositores necesitan ordenar el entrenamiento físico y registrar progreso sin perder el hilo entre apps genéricas y hojas de cálculo. Se descartó un monolito web-only porque el **uso real** es móvil (gimnasio, pista). De ahí la decisión de **cliente Android + API REST + MySQL**.

El modelo relacional se diseñó alrededor de **oposición** → **pruebas** → **baremos** y de **usuario** → **marcas** → **nivel** → **rutina**. Esta estructura permite explicar en la defensa **por qué** no basta una única tabla “entrenamientos” sin FKs: el sistema debe **validar coherencia** entre marcas, género y baremo.

22.2. Fase B — Autenticación y sesión (semanas 5–7)

La autenticación es el primer **corte vertical** serio: sin usuarios fiables, el resto de endpoints carece de sentido. Se implementó registro con hash, login con JWT y, posteriormente, **terceros** (Google/Firebase) para acercarse a un producto real. En paralelo, el cliente almacenó token en **DataStore** y se añadió la verificación `/me` para recuperar perfil tras reinicios.

22.3. Fase C — Rutinas y lógica de negocio (semanas 8–11)

Aquí concentra el “valor” del TFG: **inferir nivel**, elegir rutina oficial por enfoque y soportar **rutinas personalizadas**. La complejidad no está en Compose, sino en **reglas de negocio** y en **consistencia** con datos semilla. Por eso se documentan tablas de baremo y pruebas con claridad: un error de semilla se traduce en niveles incorrectos aunque el código sea perfecto.

22.4. Fase D — Historial, RSS y pulido (semanas 12–16)

El historial cierra el ciclo **entrenar** → **registrar** → **evolucionar**. El agregado RSS aporta valor informativo sin ser el núcleo; su riesgo es externo (red), por lo que se trató con **timeouts** y mensajes de fallo no catastróficos. El cierre del proyecto dedicó tiempo a **pruebas Jest**, recorridos manuales en emulador/dispositivo y **documentación** unificada.

23. Guía ampliada para la defensa oral (preguntas frecuentes)

Las respuestas son **orientativas**: deben adaptarse al discurso personal del alumno.

1. ¿Por qué Node.js y no PHP/Java?

Por encaje con el ecosistema npm, Express y despliegue sencillo en PaaS; el ciclo DAM no exige un lenguaje concreto si se justifica arquitectura y seguridad.

2. ¿Dónde está la lógica de negocio?

Principalmente en **servicios** del backend (`*Service.js`) y en validaciones de perfil completo antes de calcular nivel.

3. ¿Por qué JWT y no sesiones de servidor?

Escalabilidad y alineación con APIs stateless; el cliente guarda el token.

4. ¿Cómo evitas que un usuario lea datos de otro?

El token incluye identidad; las consultas filtran por `userId` del token y no por parámetros arbitrarios del cliente.

5. ¿Qué pasa si el RSS falla?

La app debe mostrar ausencia de noticias sin cerrarse; el núcleo (entrenamiento) sigue funcionando.

6. ¿Qué limitaciones admite el proyecto?

Dependencia de fuentes externas, latencia móvil, ausencia de panel admin completo, etc.

7. ¿Cómo se prueba el backend?

Jest con mocks; pruebas manuales para flujos end-to-end.

8. ¿Por qué MySQL?

Modelo relacional claro, transacciones, integridad referencial alineada con el dominio.

9. ¿Qué patrón usa Android?

MVVM con `ViewModel` + `StateFlow`, navegación con `NavHost`.

10. **¿Dónde se convierten unidades?**

En utilidades de cliente (Units.kt) y preferencias en settings.

11. **¿Cómo se calcula el nivel?**

Medias de notas por baremo y umbrales definidos en servicio; estado INCOMPLETO si faltan pruebas.

12. **¿Qué es un 409 en historial?**

Conflicto de negocio (p. ej. sesión duplicada) según implementación del servicio.

13. **¿Por qué Firebase Admin en servidor?**

Verificar idToken emitido por Firebase Auth; no basta con confiar en el cliente.

14. **¿Cómo desplegarías en producción?**

HTTPS, variables de entorno, rotación de JWT_SECRET, backups MySQL.

15. **¿Qué harías con más tiempo?**

Tests UI, internacionalización, panel admin, monitorización.

24. Competencias transversales (comunicación, aprendizaje autónomo, digitalización)

Competencia transversal	Manifestación en el TFG
Comunicación	Documentación dual (tribunal + técnica), estructura de memoria clara
Aprendizaje autónomo	Resolución de errores de integración (CORS, tokens, RSS)
Digitalización	Producto móvil + API + datos estructurados
Iniciativa	Elección de dominio realista (oposiciones) y alcance acotado

25. Indicadores de sostenibilidad del código (autoevaluación)

- **Modularidad:** separación rutas/controladores/servicios en backend; paquetes por capa en Android.
- **Legibilidad:** nombres alineados con dominio (baremos, rutinas_opo, marcas_perfil).
- **Extensibilidad:** nuevas oposiciones/pruebas cargables por datos semilla sin recompilar cliente (salvo textos hardcodeados puntuales).

- **Deuda reconocida:** tests UI pendientes; administración de datos maestros vía SQL.

Fin bloque narrativo y guía de defensa ampliada.

26. Marco normativo y formativo (Formación Profesional en España)

La Formación Profesional del sistema educativo español se orienta a la **adquisición de competencias profesionales** mediante módulos que integran conocimientos, habilidades y actitudes. En el ciclo de **Desarrollo de Aplicaciones Multiplataforma**, el proyecto final de curso tiene una función **integradora**: demostrar que el estudiante es capaz de aplicar de forma conjunta contenidos de programación, bases de datos, interfaces y despliegue, respetando buenas prácticas y documentando el proceso.

Desde la perspectiva del tribunal, interesa ver **trazabilidad** entre el enunciado o la propuesta, el producto entregado y la memoria. OpoFit facilita esa lectura al separar deliberadamente un documento **para profesorado** (énfasis en competencias, planificación y resultados) de un documento **técnico completo** (énfasis en código, API y datos). Esta duplicación controlada no es redundancia innecesaria: responde a **audiencias distintas** con expectativas de profundidad diferentes.

27. Descripción funcional de pantallas (vista tribunal)

A continuación se resume **qué ve el usuario** y **qué objetivo cumple** cada pantalla principal, sin entrar en firmas de funciones ni nombres de componentes internos. Sirve para la **demo oral** o para lectura rápida del tribunal.

Pantalla / destino	Objetivo para el usuario	Comentario pedagógico
Inicio de sesión	Entrar con email/contraseña o proveedor	Punto de aplicación de seguridad y mensajes de error comprensibles.
Registro	Crear cuenta y vincular oposición inicial	Recogida de datos físicos y marcas iniciales; validación en cliente y servidor.
Principal / Home	Acceso a módulos (rutinas, perfil, ajustes)	Navegación clara; reduce curva de aprendizaje en demo.

Pantalla / destino	Objetivo para el usuario	Comentario pedagógico
Rutinas (lista / selección)	Elegir enfoque u oposición según flujo	Conecta datos de perfil con peticiones a API.
Entrenamientos	Ver bloques de ejercicios de rutina oficial	Momento donde se evidencia la lógica de negocio "en pantalla".
Detalle de rutina / ejercicio	Instrucciones y metadatos	Integración de multimedia (URLs) si existen.
Rutinas libres	Crear y gestionar rutinas personalizadas	Muestra CRUD más allá del "camino feliz" de datos precargados.
Crear rutina	Seleccionar ejercicios del catálogo	Evalúa usabilidad en listas largas y formularios.
Registrar entrenamiento	Volcar resultados por ejercicio	Punto crítico de integridad: coherencia con historial.
Historial / evolución	Ver progresión	Traduce datos en valor percibido por el opositor.
Perfil / editar perfil	Ajustar marcas y datos físicos	Vinculación directa con baremos y nivel.
Información de oposición	Textos, requisitos, noticias	Combinación de datos estáticos + RSS.
Ajustes	Unidades y preferencias	Demuestra persistencia local y consistencia con API.

28. Rúbrica extensa de evaluación (orientación para el tribunal)

La siguiente tabla propone **criterios** y **niveles** descriptivos. El centro puede adaptar pesos; aquí solo se ofrece una malla coherente con un proyecto full-stack móvil.

28.1. Criterio: Análisis y diseño

Nivel	Descriptores
Inicial	Requisitos confusos; modelo de datos incompleto o inconsistente.

Nivel	Descriptores
En desarrollo	Requisitos enumerados; ER básico con principales entidades.
Competente	Modelo normalizado; relaciones justificadas; casos de uso claros.
Avanzado	Reflexión sobre decisiones (por qué JWT, por qué tablas de unión, etc.).

28.2. Criterio: Implementación backend

Nivel	Descriptores
Inicial	Mezcla de lógica en rutas; errores frecuentes 500 sin manejo.
En desarrollo	Separación parcial; validaciones mínimas.
Competente	Capas ruta–controlador–servicio; códigos HTTP coherentes; JWT y límites.
Avanzado	Pruebas automáticas; consideraciones de seguridad y despliegue.

28.3. Criterio: Cliente Android

Nivel	Descriptores
Inicial	UI fragmentada; estado inconsistente; crashes frecuentes.
En desarrollo	Navegación funcional; errores de red poco claros.
Competente	MVVM; feedback de carga y error; persistencia de sesión razonable.
Avanzado	Tema y accesibilidad básicos cuidados; código organizado por paquetes.

28.4. Criterio: Documentación y comunicación

Nivel	Descriptores
Inicial	Memoria incompleta; sin instrucciones de despliegue.
En desarrollo	README mínimo; capturas sueltas.
Competente	Memoria estructurada; API descrita; guía de instalación.
Avanzado	Dos niveles de documentación (tribunal + técnica); trazabilidad requisito–código.

29. Bibliografía y fuentes comentadas (ampliada)

1. **Documentación oficial Android Jetpack (Compose, Navigation, DataStore).** Consulta imprescindible para justificar decisiones de UI y estado; permite al tribunal contrastar el uso de APIs modernas frente a soluciones obsoletas.
 2. **Express.js (referencia de routing y middleware).** Fundamenta el diseño de capas y el orden de rutas parametrizadas (conflictos `/:id` vs rutas estáticas).
 3. **MySQL Reference Manual.** Apoyo para explicar tipos ENUM, InnoDB y restricciones FK en defensa.
 4. **RFC 7519 (JWT, visión general).** No es necesario memorizar el estándar, pero sí comprender roles de cabecera, payload y firma.
 5. **Material Design 3.** Coherencia visual; el tribunal valora presentación cuidadosa aunque el peso principal sea técnico.
 6. **OWASP (buenas prácticas mínimas).** Referencia para hablar de contraseñas, tokens y exposición de secretos sin entrar en auditoría profesional.
 7. **Documentación Firebase Admin SDK.** Justifica la verificación de tokens en servidor frente a clientes manipulables.
 8. **Especificaciones de oposiciones (fuentes públicas).** Contextualizan el dominio; el TFG no valida legalmente convocatorias, pero sí muestra alineación con pruebas físicas reales a nivel informativo.
-

30. Anexo: guion sugerido de demostración (15–20 minutos)

1. **Introducción (1 min):** problema, usuarios objetivo, arquitectura en una frase (cliente-API-BD).
 2. **Registro / login (2–3 min):** mostrar validaciones y mensaje de error creíble.
 3. **Perfil y marcas (3 min):** explicar cómo las marcas alimentan nivel y rutina.
 4. **Rutina oficial (4 min):** seleccionar enfoque; mostrar lista de ejercicios y descansos.
 5. **Rutina libre (3 min):** crear una rutina corta desde catálogo.
 6. **Historial (2 min):** registrar sesión y mostrar evolución.
 7. **Oposición + noticias (2 min):** RSS o mensaje controlado si falla la red.
 8. **Cierre (1 min):** pruebas Jest, limitaciones, trabajo futuro.
-

31. Relación explícita con el documento técnico completo

El lector del presente documento puede profundizar en:

- Diccionario de tablas SQL y manual de endpoints (Documentacion_Completa_OpoFit.md, secciones 55–57 del índice).
- Fichas por archivo Kotlin en la memoria técnica (revisión línea a línea del cliente).
- Inventario de backend y dependencias entre servicios (secciones 50–54 del índice en la memoria técnica).

Esta separación permite que el tribunal **no pierda el bosque por el árbol**: primero valora el proyecto como conjunto; después puede bajar al detalle si lo desea.

Fin del bloque de volumen académico (secciones 29–32).

32. Diario de trabajo por semanas (plantilla desarrollada)

La siguiente tabla resume **actividades** y **resultados** semana a semana. Las cifras de horas son **orientativas**; el alumno puede sustituirlas por las reales exigidas por el centro.

Semana	Actividades principales	Resultado tangible	Horas aprox.
1	Lectura de requisitos, boceto de casos de uso, decisión de stack	Documento de alcance v0.1	8–12
2	Diseño ER en papel/herramienta; revisión de normalización	Esquema mydb borrador	10–14
3	Implementación DDL; corrección de FK; primera carga seed	BD importable sin errores	10–16
4	Proyecto Express mínimo; health-check; CORS	Servidor escuchando en puerto	8–12
5	Registro/login; bcrypt; JWT; pruebas Postman	Flujo auth en API	14–20
6	Middleware token; rutas usuario; ajustes	CRUD perfil básico	12–18

Semana	Actividades principales	Resultado tangible	Horas aprox.
7	Servicio de rutinas; consultas SQL complejas	Endpoint mi-entrenamiento	16–22
8	Cliente Android: login, tema, navegación mínima	APK con pantallas auth	14–20
9	Integración Retrofit; manejo de errores HTTP	Llamadas reales a API	12–18
10	Pantallas de rutinas y entreno; estado en ViewModel	Flujo usable en emulador	16–24
11	Rutinas personalizadas + catálogo ejercicios	Creación y listado persistidos	14–20
12	Historial y evolución; gráficas o listas	Registro de sesión completo	14–20
13	RSS; pantalla de información de oposición	Noticias o mensaje de fallo controlado	10–16
14	Pulido UI; modo oscuro; unidades	Coherencia visual	10–14
15	Jest ampliado; corrección de bugs; regresión manual	Suite verde en npm test	12–18
16	Memoria unificada; exportación PDF; ensayo defensa	Paquete de entrega	16–24

32.1. Reflexión sobre el reparto de esfuerzo

Las semanas **7–8** y **10–12** concentran la mayor carga porque combinan **SQL**, **lógica de negocio** y **UI**. Es normal que el estudiante solicite tutorías en esos puntos: no indica fallo de planificación, sino **punto de máxima integración**.

33. Cuestionario extenso para preparar la defensa (respuestas modelo)

Las respuestas son **guías**; deben personalizarse.

P1. ¿Cuál es la aportación original de OpoFit frente a una app de running genérica?

OpoFit ancla el entrenamiento en el **baremo y las pruebas oficiales** de una oposición concreta, no solo en volumen de kilómetros. La lógica de nivel y rutina emerge de **datos tabulados** (marcas, género, oposición), no de un plan fijo para cualquier usuario.

P2. ¿Por qué separar Documentacion_Profesores y Documentacion_Completa?

Para respetar dos lecturas: el tribunal necesita **contexto y competencias** sin 200 páginas de fichas de código; el desarrollador o el evaluador técnico necesita **trazabilidad fina** y referencias de API.

P3. ¿Dónde estaría el cuello de botella si miles de usuarios usaran la app?

Probablemente la **base de datos** (consultas de baremo y rutina) y el **pool de conexiones**. Habría que medir con profiling, índices adicionales, caché de lectura para catálogos y escalado horizontal del backend.

P4. ¿Cómo se protege la contraseña?

Con **bcrypt** en registro; nunca se almacena en claro. El cliente nunca recibe el hash en respuestas de login exitoso como dato a mostrar.

P5. ¿Qué ocurre si manipulo el userId en el JSON del cliente?

El servidor debe comparar con el **id del token**. Los tests de UsuarioController cubren explícitamente el **403** cuando no coinciden.

P6. ¿Por qué ENUM en género y nivel?

Para **restringir** valores al dominio y alinear con datos de baremo y rutinas; reduce errores de cadena libre.

P7. ¿Es escalable el agregado RSS?

Como diseño actual, es adecuado para volumen académico. En producción habría que cachear, programar refrescos y manejar **ETags** o límites de frecuencia a sitios externos.

P8. ¿Qué métricas demostrarías en la defensa?

Número de tests Jest pasando, cobertura si se activa, tiempo de respuesta medio en local para endpoints críticos (medición informal con Postman), y número de pantallas recorridas en demo sin crash.

P9. ¿Cómo versionarías la API en el futuro?

Prefijo `/api/v2`, convivencia temporal de versiones, deprecación documentada. En el TFG se dejó `/api` único por simplicidad.

P10. ¿Qué has aprendido sobre Compose?

Que el estado debe ser **predecible** y elevarse al ViewModel cuando cruza pantallas; que la

recomposición exige listas estables (key) y evitar trabajo pesado en la UI thread.

P11. ¿Habría alternativa a MySQL?

PostgreSQL sería igualmente válido. Una NoSQL puro sería peor encaje para integridad relacional fuerte entre baremos y pruebas.

P12. ¿Cómo documentarías un bug encontrado tarde?

Issue con pasos de reproducción, captura de log del servidor, commit del fix y referencia en memoria bajo "Incidencias resueltas".

P13. ¿Qué significa "estado INCOMPLETO" en el negocio?

Que el usuario no ha registrado marcas para **todas** las pruebas exigidas por su oposición; el sistema **no inventa** un nivel hasta completar datos mínimos.

P14. ¿Cómo garantizas que una rutina personalizada es del usuario?

Por userId en token al listar/eliminar y por FK en tablas; los tests refuerzan el 403 ante suplantación básica.

P15. ¿Qué rol juega seed.sql?

Permite demostración **reproducible** en cualquier máquina del tribunal si se dispone de MySQL; contiene datos coherentes con las pantallas.

P16. ¿Por qué DataStore y no SharedPreferences?

API moderna basada en corrutinas y flujos; menos bloqueo en hilo principal y mejor alineación con Jetpack.

P17. ¿Qué harías con internacionalización?

Extraer strings a strings.xml, soportar Locale, y no hardcodear textos de error del servidor si deben mostrarse crudos (idealmente códigos + mapa en cliente).

P18. ¿Cómo explicas el uso de Firebase junto a JWT propio?

Firebase resuelve **identidad** del proveedor; el backend sigue emitiendo **JWT de aplicación** para autorizar el resto de endpoints con un modelo unificado.

P19. ¿Dónde está el "corazón" del proyecto?

En **RutinasService** y tablas de baremo/rutina: concentra la promesa de valor al usuario.

P20. ¿Qué limitación ética debe mencionarse?

Que las recomendaciones son **orientativas**; no sustituyen valoración médica ni entrenador personal certificado.

Fin diario y cuestionario extendido.

34. Léxico inglés-español (términos del proyecto)

Mucha documentación técnica está en inglés. La siguiente tabla facilita la **correspondencia** oral en la defensa.

Inglés	Español (uso en el TFG)
Backend	Servidor o capa de servidor
Frontend / client	Cliente (aquí: app Android)
Endpoint	Punto de acceso HTTP concreto (ruta + método)
Middleware	Intermediario Express (ej. validación JWT)
Token / Bearer	Token de portador en cabecera Authorization
Hash	Resumen criptográfico (contraseña con bcrypt)
Pool (connections)	Conjunto reutilizable de conexiones a BD
Migration	Migración de esquema (no usada formalmente aquí; DDL manual)
Seed data	Datos semilla / de demostración
DTO	Objeto de transferencia de datos (data class Gson)
Coroutine	Corrutina Kotlin
StateFlow	Flujo de estado observable en MVVM
Composable	Función UI en Jetpack Compose
Theme / Dark mode	Tema visual / modo oscuro
Rate limit	Límite de frecuencia de peticiones
RSS feed	Fuente de sindicación de noticias

35. Comparativa de alternativas tecnológicas (y decisión)

Alternativa	Ventaja	Inconveniente	Por qué no se eligió (o qué rol tiene)
PHP + Laravel	Madurez en hosting compartido	Menos alineado con el stack móvil del ciclo	Se priorizó ecosistema npm + Express
Spring Boot (Java)	Robustez enterprise	Mayor ceremonia para un TFG acotado	Kotlin en cliente ya ocupa tiempo; Node acelera iteración API
Firebase solo (BD + auth)	Velocidad inicial	Menos control del modelo relacional complejo	Se necesitaba SQL explícito para baremos y FK
SQLite local únicamente	Sin servidor	No cumple competencia de servicios en red como núcleo	La API es requisito central
GraphQL	Consultas flexibles	Complejidad extra para CRUD acotado	REST + JSON suficiente
SharedPreferences	Simplicidad	API bloqueante / menos moderna	DataStore preferido

36. Figuras y anexos gráficos recomendados para la memoria impresa

Aunque este PDF de profesores es **mayormente textual**, se recomienda incorporar en la versión final para el centro:

1. **Diagrama de despliegue** (móvil ↔ API ↔ MySQL ↔ proveedores externos).
2. **Diagrama entidad-relación** simplificado (solo entidades nucleares).
3. **Capturas** de login, rutina, historial y error de red.
4. **Gráfica de evolución** exportada desde la app (si el centro valora resultado visual).
5. **Tabla resumen de endpoints** ya presente en la documentación técnica, opcionalmente como figura a página completa.

Cada figura debe llevar **pie** numerada y referencia en el índice de figuras si el centro lo exige.

37. Mensaje final para el tribunal (lectura rápida)

OpoFit pretende ser un **ejemplo integrador** de competencias de un ciclo DAM: datos bien modelados, servicios accesibles de forma segura y un cliente móvil que traduce el esfuerzo físico del usuario en pantallas claras. Las imperfecciones reconocidas (tests UI, panel admin) son oportunidades de mejora explícitas, no omisiones silenciosas. Gracias por el tiempo dedicado a la lectura y la evaluación.

Fin léxico, comparativas y recomendaciones gráficas.

38. Microguía: 24 preguntas breves con respuesta en una frase

#	Pregunta	Respuesta modelo (una frase)
1	¿Qué problema resuelve OpoFit?	Ordena el entrenamiento físico de opositores y registra progreso con datos oficiales.
2	¿Backend en qué lenguaje?	JavaScript sobre Node.js con framework Express.
3	¿Base de datos?	MySQL relacional con InnoDB y claves foráneas.
4	¿Cliente?	Android nativo con Kotlin y Jetpack Compose.
5	¿Autenticación principal?	JWT en cabecera Authorization tras login.
6	¿Hash de contraseña?	bcrypt en registro; no se guarda texto plano.
7	¿Dónde está la lógica de nivel?	Principalmente en servicios del backend que leen baremos.
8	¿Rutina personalizada dónde se guarda?	Tablas rutinas_pers y detalle_rutina_pers.
9	¿Historial?	historial_sesiones y registro_resultados.
10	¿RSS para qué?	Agregar noticias públicas de oposición sin editarlas a mano.
11	¿Por qué tests Jest?	Regresión automática en controladores y middleware.
12	¿MVVM dónde?	ViewModels en Android separan UI de datos.

#	Pregunta	Respuesta modelo (una frase)
13	¿Token en cliente?	DataStore vía TokenManager.
14	¿Qué es un 401?	No autorizado: token ausente o inválido.
15	¿Qué es un 403?	Prohibido: identidad no coincide con recurso pedido.
16	¿Qué es un 409?	Conflicto de negocio (p. ej. duplicado).
17	¿Rate limiting?	Limita peticiones para proteger autenticación.
18	¿Firebase para qué?	Verificar identidad del proveedor; el backend emite JWT propio.
19	¿Despliegue backend?	Cualquier PaaS con Node; variables de entorno para secretos.
20	¿HTTPS?	Obligatorio en producción real; en local suele ser HTTP.
21	¿Principal riesgo del dominio?	Datos de baremo desactualizados respecto a convocatoria real.
22	¿Trabajo futuro prioritario?	Tests UI y panel de administración de datos maestros.
23	¿Documentación dual para qué?	Tribunal vs detalle técnico sin mezclar audiencias.
24	¿Cómo se llama el paquete Android?	com.opofit.miapp.

Fin microguía.

39. Conclusión académica

El proyecto **OpoFit** integra de forma conjunta un conjunto representativo de tecnologías del stack DAM actual: **API REST segura, persistencia relacional, cliente móvil moderno y servicios de terceros**. La separación en capas facilita el mantenimiento y la evaluación por rúbricas técnicas. La **documentación técnica detallada** se concentra en **Documentacion_Completa_OpoFit.md**, donde se audita la trazabilidad entre requisitos, código y despliegue.

40. Referencias técnicas (consulta)

- Documentación Express 5 y path-to-regexp (rutas parametrizadas).
 - MySQL 8 — referencia de InnoDB y restricciones FK.
 - Android Developers — Jetpack Compose, Navigation Compose, DataStore.
 - Firebase — verificación de ID tokens en servidor con Admin SDK.
 - RFC 7519 — JSON Web Token (conceptos generales).
-

41. Cómo se complementan los dos documentos

Documento	Para quién lo planteé	Qué cubre sobre todo
Documentacion_Profesores_OpoFit.md (este)	Tribunal y tutoría	Competencias, planificación, metodología, riesgos, resultados
Documentacion_Completa_OpoFit.md	Revisión técnica fina	Base de datos, API, pantallas, anexos con tablas SQL, JSON de ejemplo y fichas por archivo Kotlin

En la entrega presento **los dos**: primero la lectura corta para la defensa oral y, como soporte, la memoria larga donde está el detalle que no cabe en un discurso de quince minutos.

Fin de la memoria para el tribunal — OpoFit